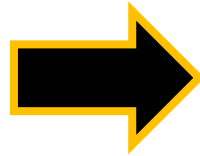


6.3 이미지 분할을 위한 신경망

목차

- 이미지 분할이란?
- 이미지 분할의 쓰임새
- 완전 합성곱 네트워크
- 합성곱 & 역합성곱 네트워크
- U-Net
- PSPNet
- DeepLabv3 / DeepLabv3+
- U-Net 실습

이미지 분할이란?



이미지 분할(Image Segmentation)은 신경망을 훈련시켜 이미지를 픽셀 단위로 분할 하는 것을 의미
즉, 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출하는 것

대표적인 이미지 분할 네트워크로는 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크,
U-Net, PSPNet, DeepLabv3/ DeepLabv3+가 존재

이미지 분할의 쓰임새

의료 및 헬스케어

MRI나 CT 영상에서 종양, 장기, 혈관 등의 위치를 정확히 구분하여 의사의 진단과 수술 계획에 도움을 줌

자율주행 및 로봇 공학

차량 카메라에 입력된 화면에서 도로, 인도, 보행자, 신호등을 실시간으로 분할 인식하여 안전한 주행 경로를 확보

미디어 및 사진 편집

배경을 흐리게 처리하거나, 영상 합성 및 크로마키 작업을 할 때 특정 객체만 깔끔하게 따내는 데 사용

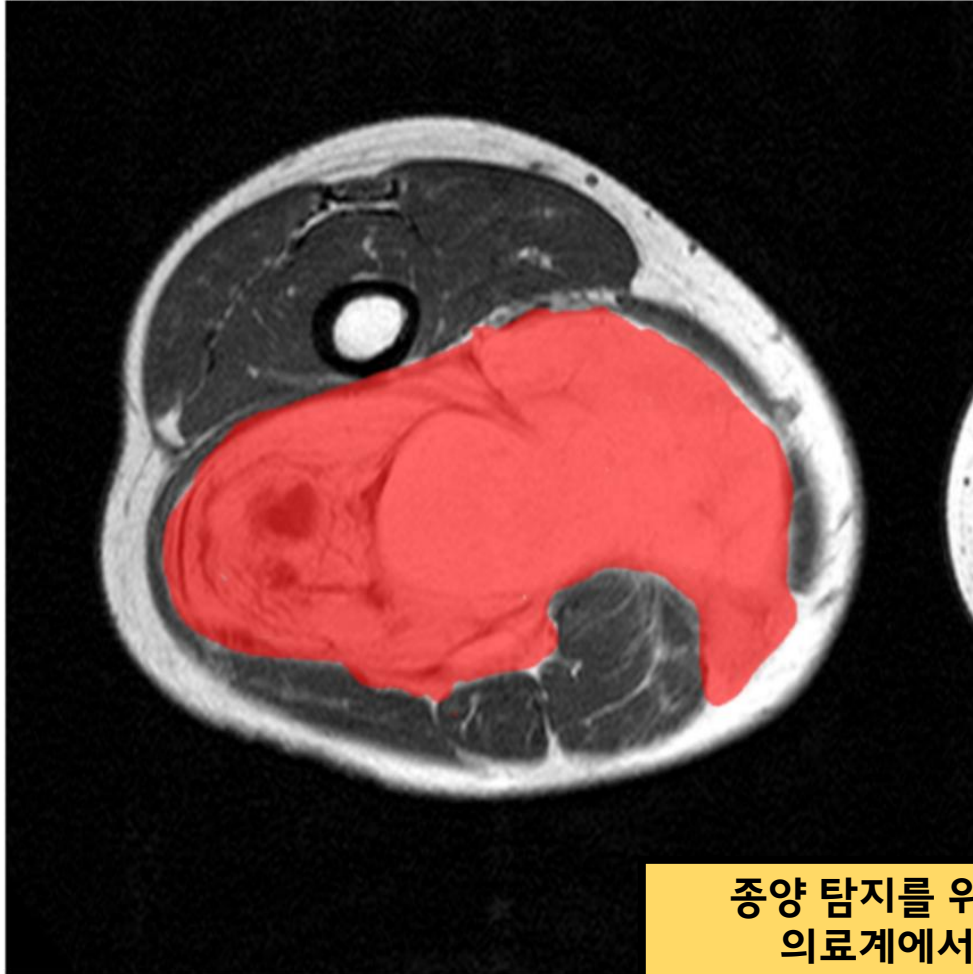
스포츠 분석

경기 영상에서 선수의 신체 부위나 공, 장비의 위치를 픽셀 단위로 추적하여, 전술적 위치 선정, 정밀 판정에 활용

산업 및 제조

제품 표면의 스크래치, 균열, 이물질 등의 결함 부위를 정확히 분할해내어 불량품을 자동으로 탐지

이미지 분할의 쓰임새

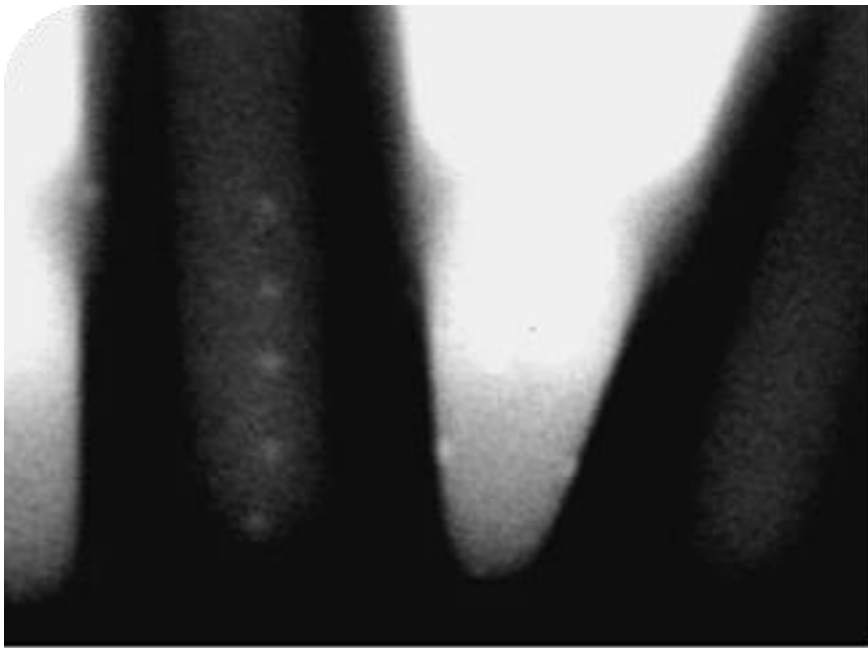


종양 탐지를 위해
의료계에서
이미지 분할 기술을 사용



자율 주행 시 대상의 위치와
영역을 수집하기 위해
이미지 분할 기술을 사용

이미지 분할의 쓰임새



Original Image



Instance Segmentation

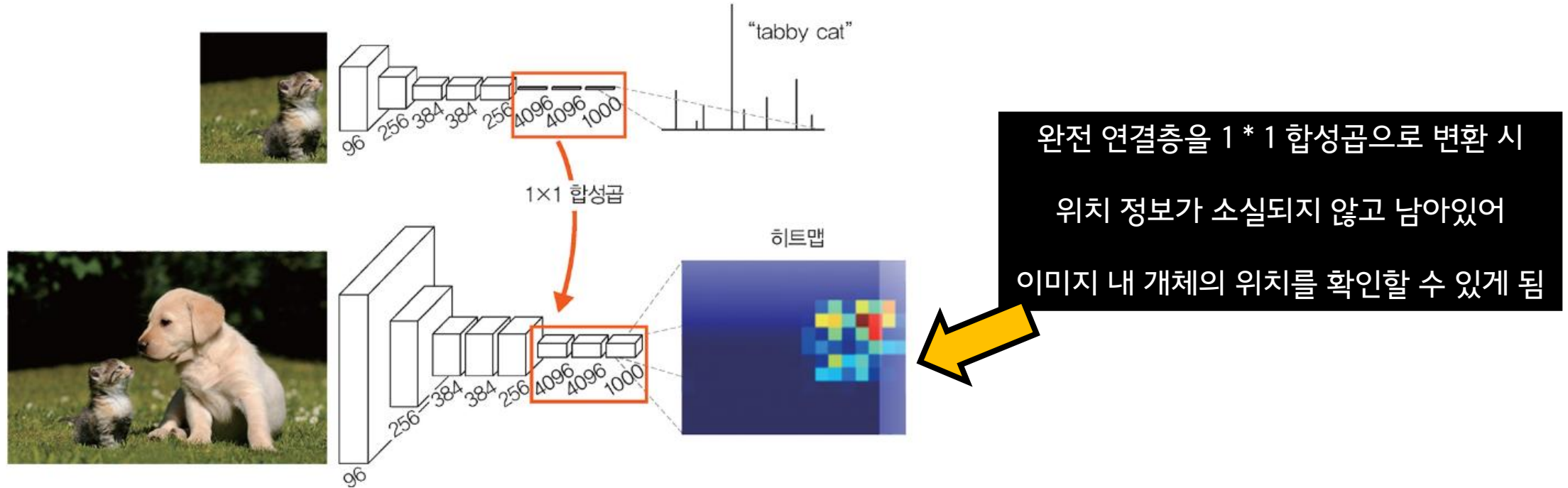
공장 생산 라인에서
미세한 결함을 찾아내고
시각화하여 불량품을 탐지

이미지 분할의 쓰임새

축구 경기를 분석하여
실시간으로 대형과 전술을 분석하고
이를 이미지 분할 기술로 시각화



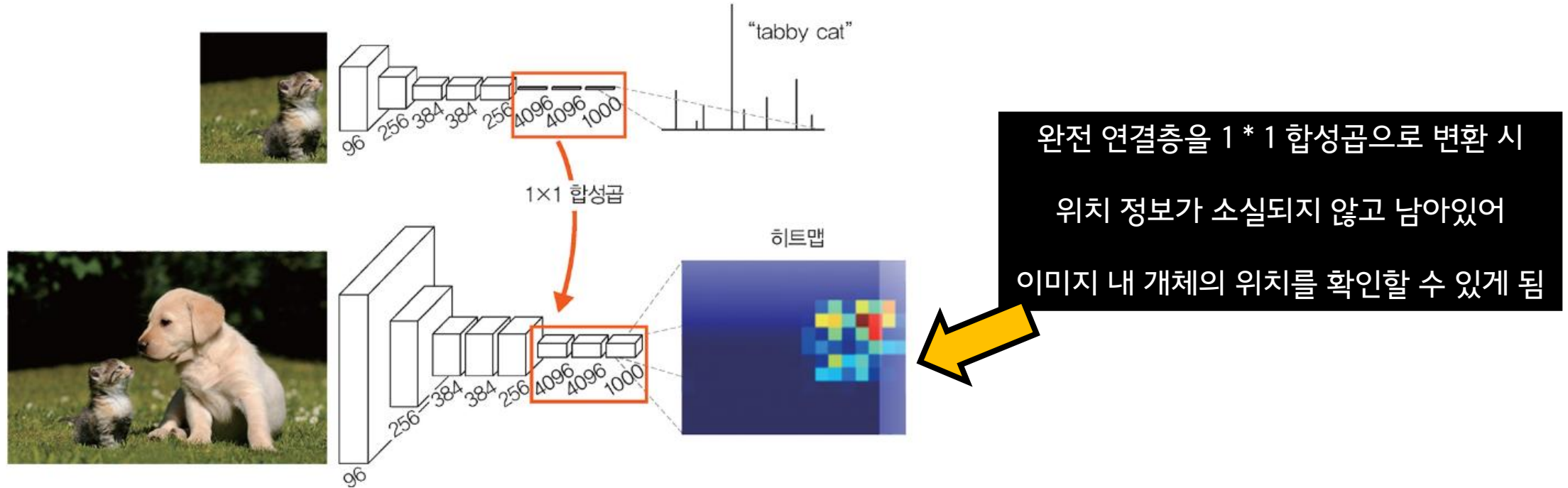
완전 합성곱 네트워크 소개



완전연결층은 고정된 크기의 입력만 받아들이며, 완전연결층을 거친 후에는 위치 정보가 사라진다는 한계가 존재

이를 해결하기 위해 완전연결층을 1 * 1 합성곱으로 대체하는 것이
완전 합성곱 네트워크 (Fully Convolutional Network, FCN)

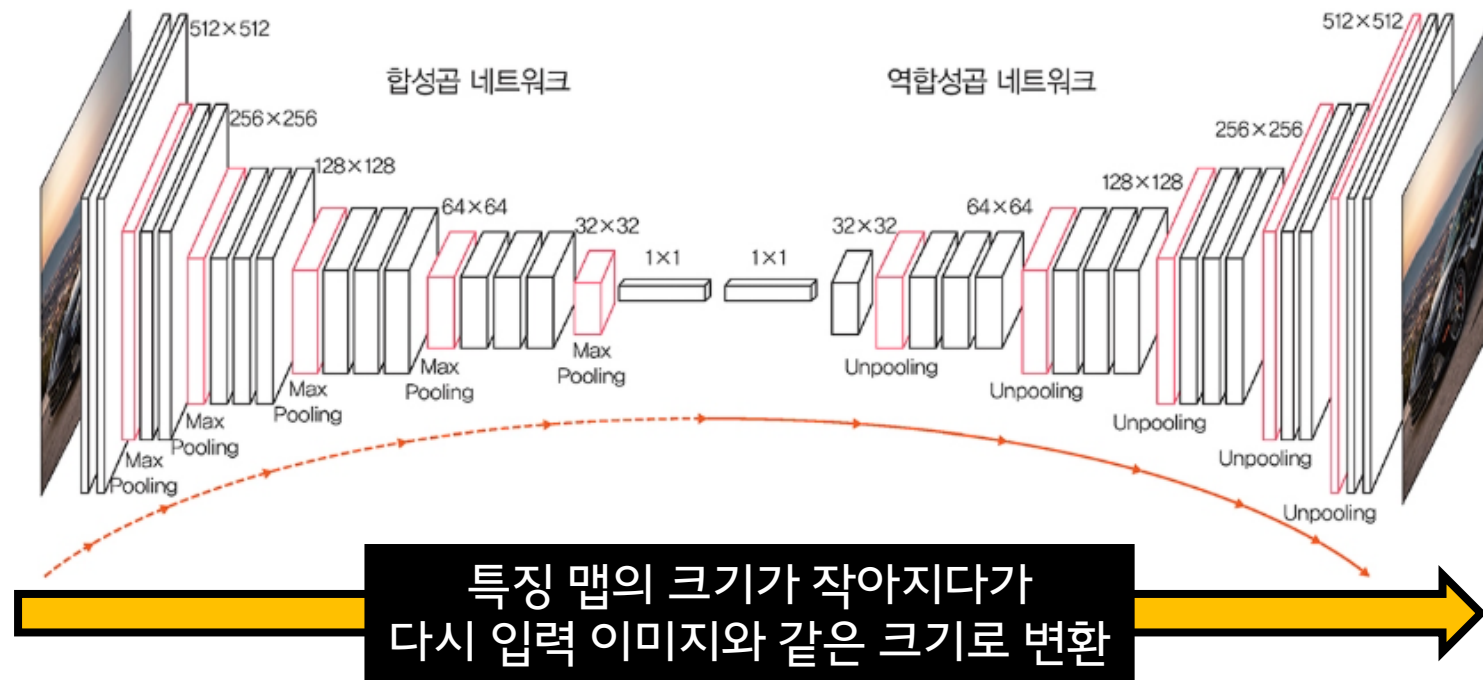
완전 합성곱 네트워크 소개



완전 합성곱 네트워크 (Fully Convolutional Network, FCN)는 위치 정보가 보존되지만, 다음과 같은 단점을 가짐

- 여러 단계의 합성곱과 풀링층을 거치며 해상도가 낮아짐
- 낮아진 해상도를 복원하기 위해 업 샘플링을 진행하여, 이미지의 세부 정보를 잃게 됨

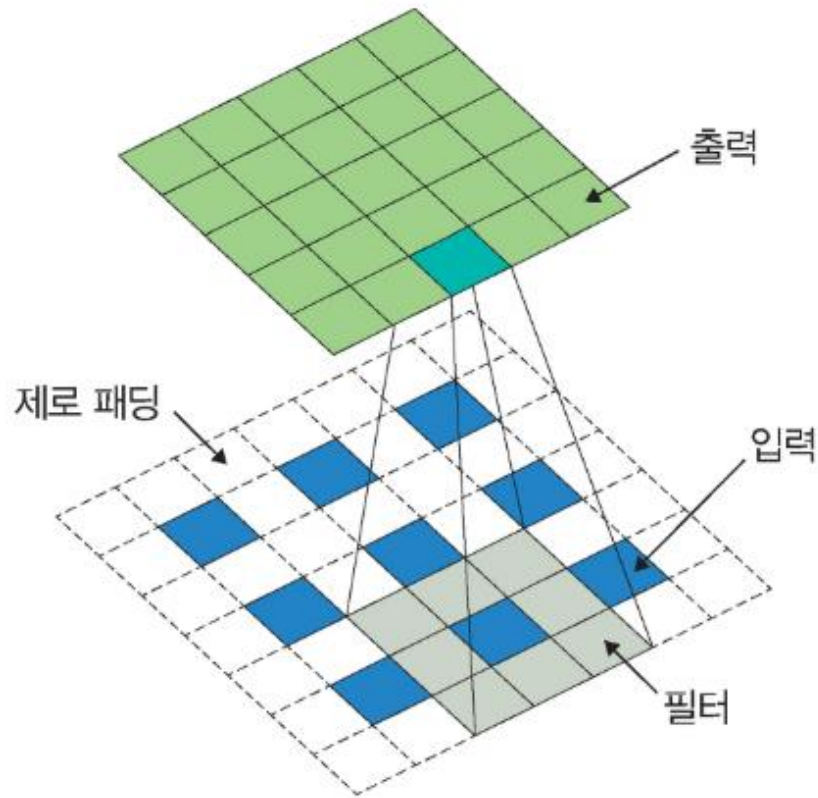
합성곱 & 역합성곱 네트워크 소개



완전 합성곱 네트워크 (Fully Convolutional Network, FCN)의 단점을 해결하기 위해
역합성곱 네트워크를 도입한 것이

합성곱 & 역합성곱 네트워크 (Convolutional & Deconvolutional Network)

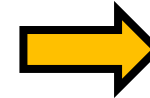
역합성곱이란?



1. 각각의 픽셀 주위를 0으로 채우는 **제로 패딩**을 진행
2. 패딩된 맵 위에 합성곱 연산을 수행

제로 패딩(Zero Padding)

1

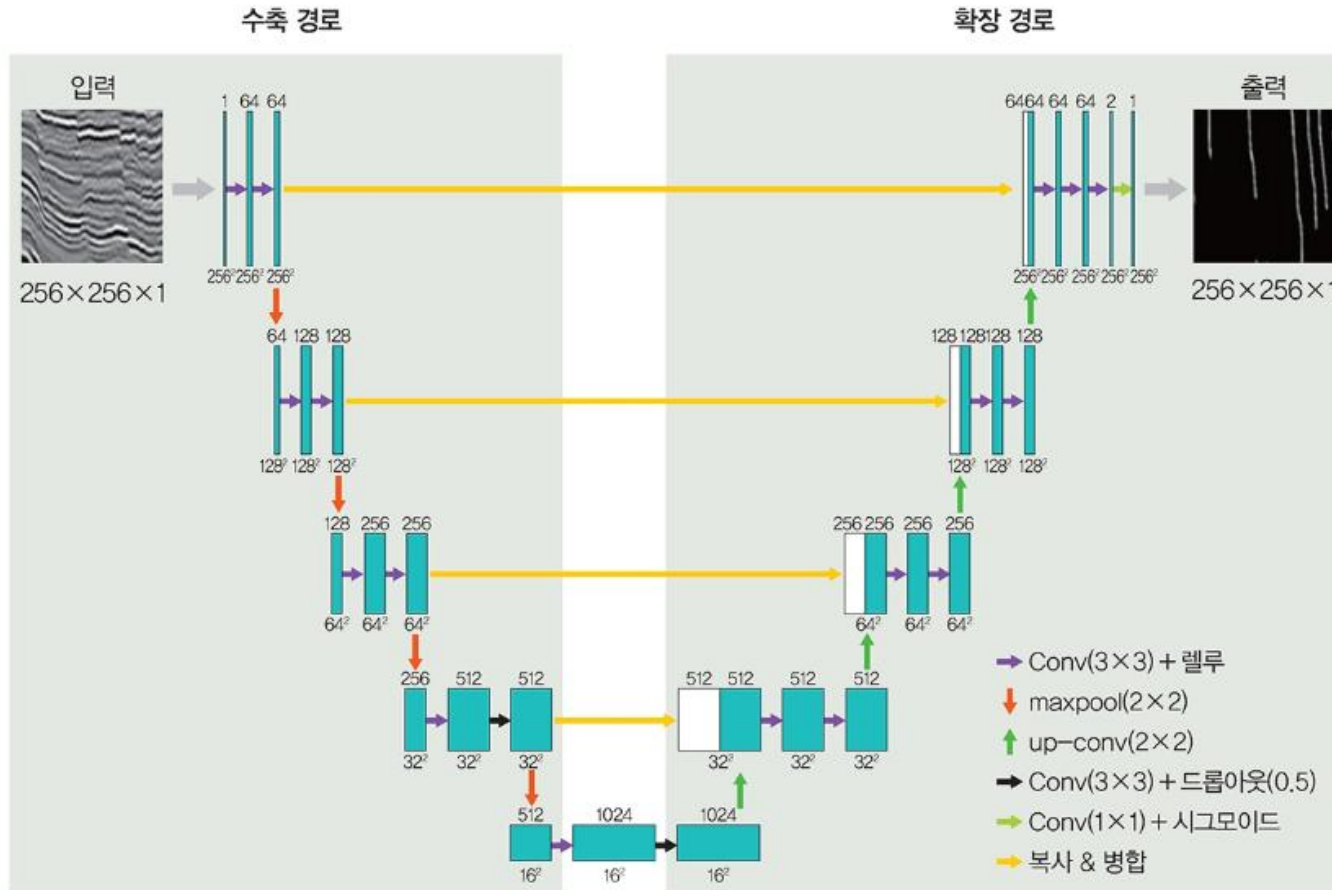


0	0	0
0	1	0
0	0	0

역합성곱 (Deconvolution)은 업샘플링 (Upsampling)이라고도 함

역합성곱은 합성곱 기반 신경망의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용
기본 합성곱이 합성곱 연산을 통해 특성 맵의 크기를 줄여갔다면,
역합성곱은 이와 반대로 특성 맵 크기를 증가시키는 방식으로 동작

U-Net 소개



- U-net은 FCN 기반으로 구축됨
- 수축 경로와 확장 경로로 구성
- 수축 경로는 맥락(Context)를 포착
- 확장 경로는 특성 맵을 업샘플링
- 이후 수축 경로에서 포착한 특성 맵의 맥락(Context)과 결합하여 **지역화**를 수행

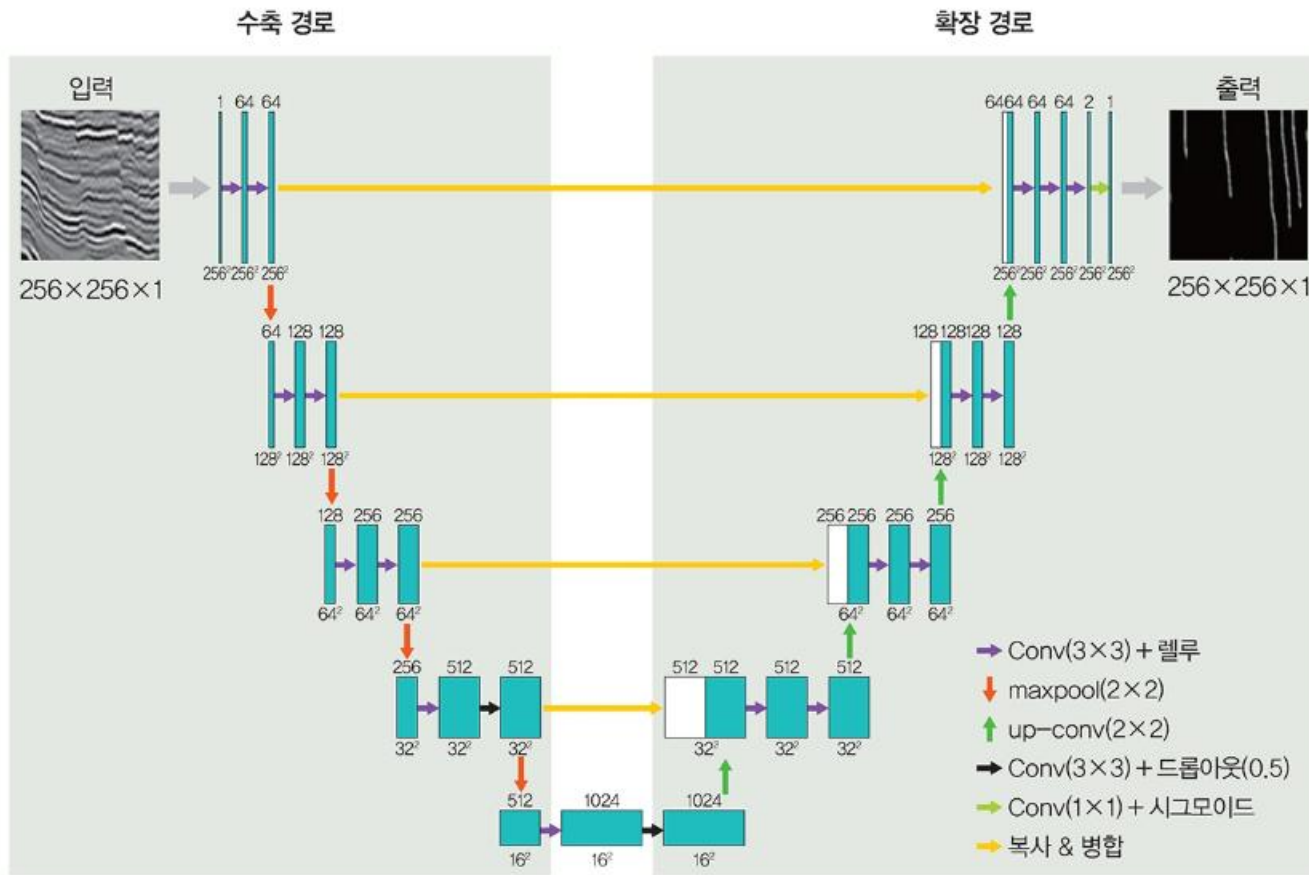


지역화
(Localization)
이미지 안에 객체의
위치 정보를 출력해주는 것
경계 상자(Bounding Box)를
이용해 주로 표현.
경계상자의 경우,
왼쪽 위, 오른쪽 아래 좌표로 표현

U-Net은 바이오 메디컬 이미지 분할을 위해 제시되었던 합성곱 신경망

기존 슬라이딩 윈도우 방식과는 다르게 검증이 끝난 부분을 다음 패치에서 건너뛰기 때문에 속도가 빠름

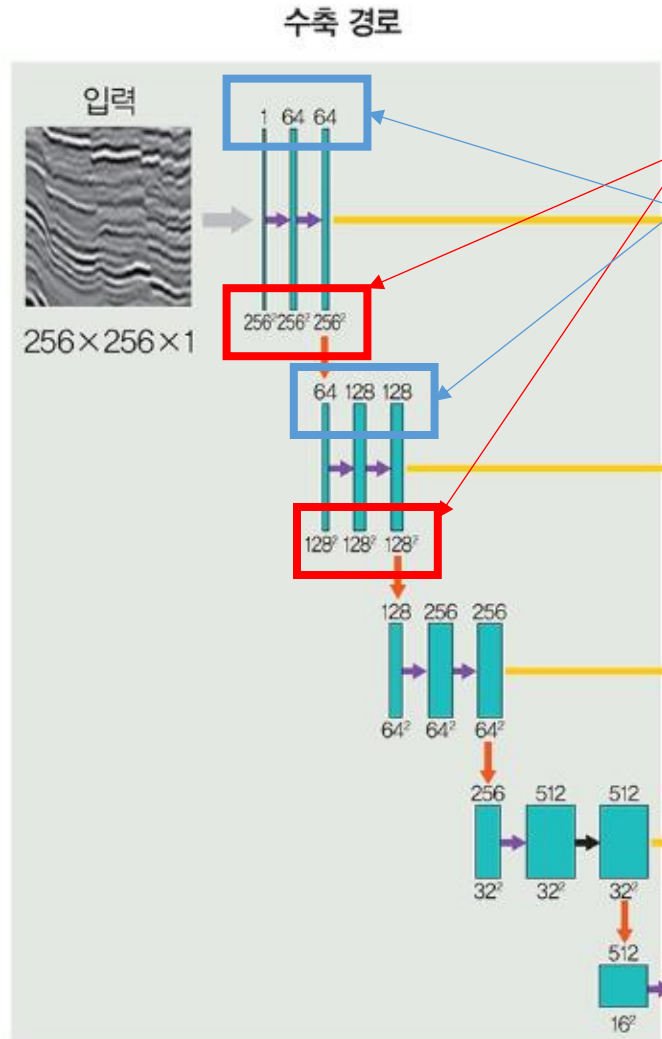
U-Net 전체 구조



- 3 * 3 합성곱 두 개를 하나로 묶은 합성곱 블록 사용
- 두 합성곱 사이에 Dropout을 배치
- 수축 경로에서는 Max Pooling을 사용해 크기를 줄임
- 확장 경로에서는 합성곱 블록 앞에 up-conv를 추가
- 수축 과정에서 줄어든 크기를 다시 키우기 위함

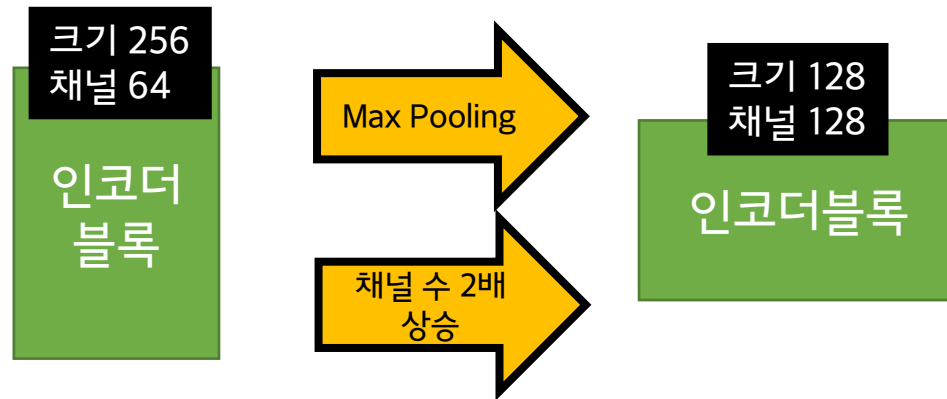
즉, U-Net은 크기가 다양한 이미지의 객체를 분할하기 위해
크기가 다양한 특성 맵을 병합할 수 있도록
다운 샘플링과 업 샘플링을 순서대로 반복하는 구조로 구성됨

U-Net 인코더 (수축 경로)



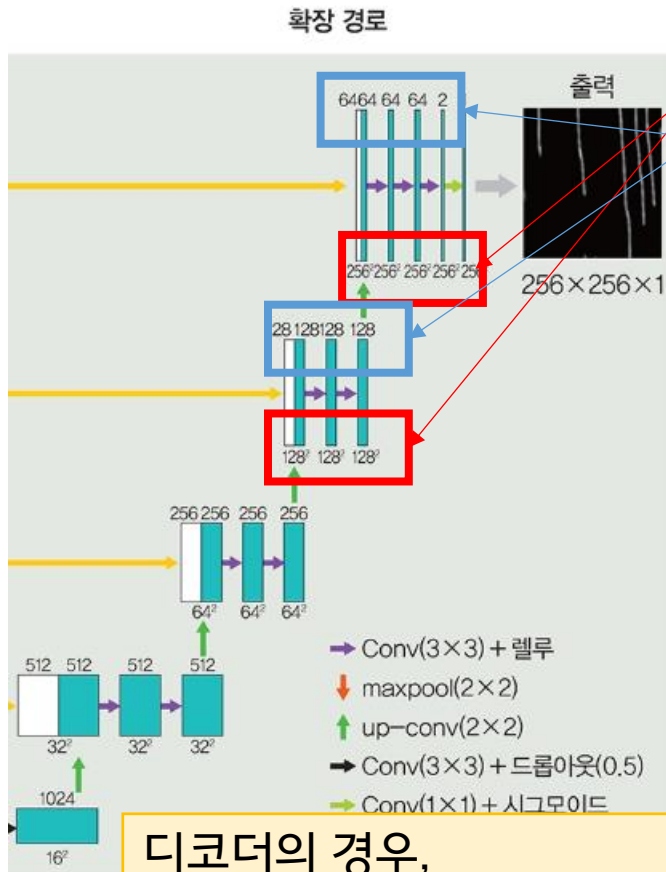
각각의 인코더 블록을 통과할 때마다
해상도는 1/2배로,
채널 수는 2배로 변경

즉 “크기”는 반으로 줄이되 특징만 압축하여 그 “밀도”를 2배로 올리는 역할



인코더의 경우,
특징 추출(Feature Extraction)단계와
공간적 크기 축소(Spatial Downsampling) 단계로 나뉘어
입력한 이미지의 특징 데이터를 추출하며 크기를 줄이는 것을 목적으로 함

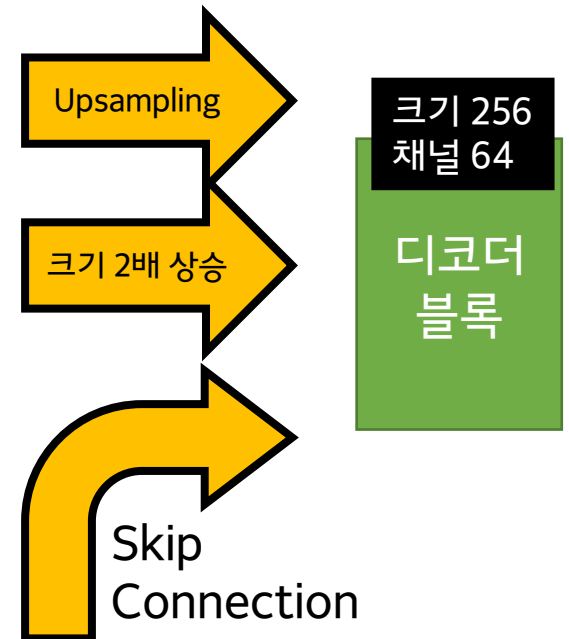
U-Net 디코더 (확장 경로)



각각의 디코더 블록을 통과할 때마다
해상도는 2배로,
채널 수는 1/2배로 변경
단순히 크기와 채널만 변경하는 것이 아니라,
디코더 블록에 대응되는 인코더 블록의 사전 정보를 결합
(스킵 커넥션 결합)

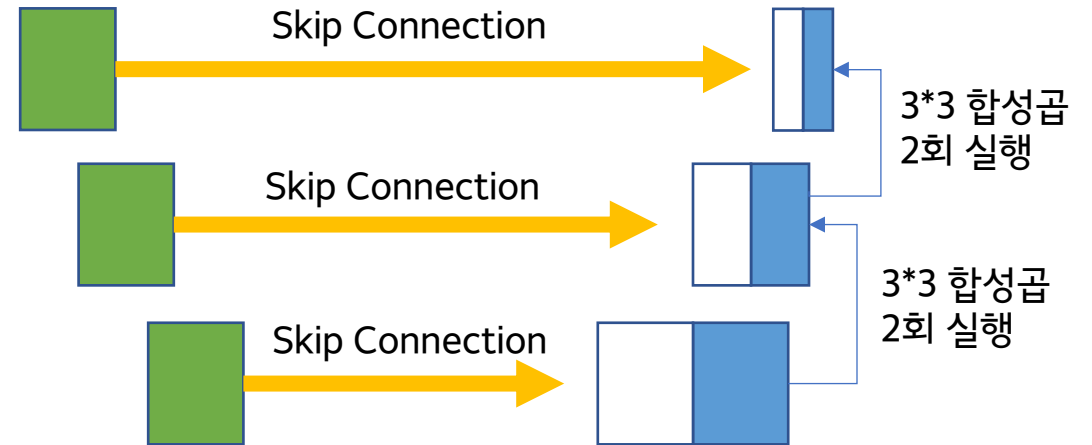
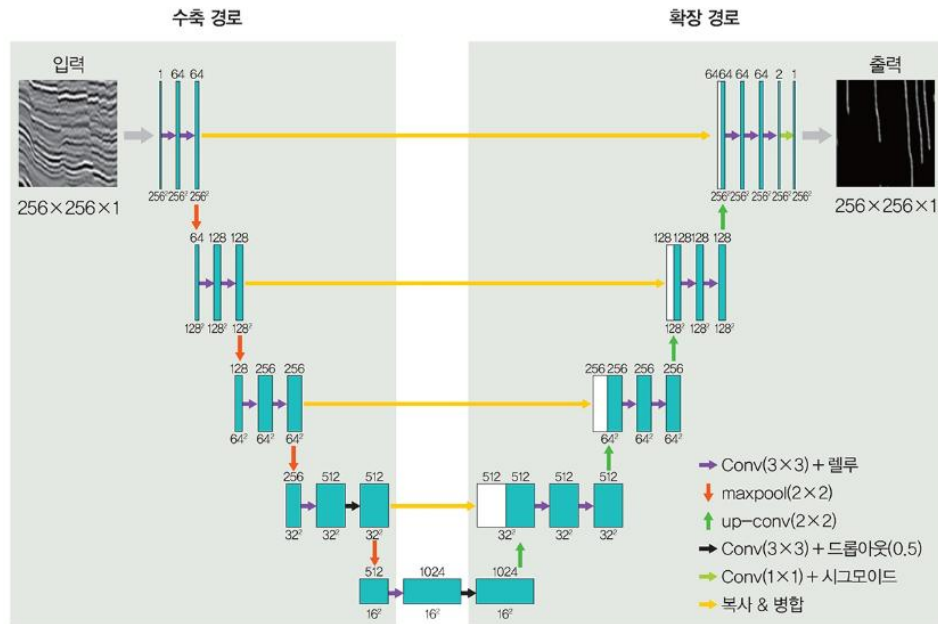
즉 “밀도”는 반으로 줄이되
“크기”를 2배로 올려 원본 이미지와
같은 크기를 만드는 역할

크기 128
채널 128
디코더블록



디코더의 경우,
인코더가 만든 특징 정보를 기반으로, Skip Connection을 통한
원본 정보를 참고하며 원래 이미지 크기의 특징 맵을 만드는 것을 목적으로 함

U-Net 스킵 커넥션

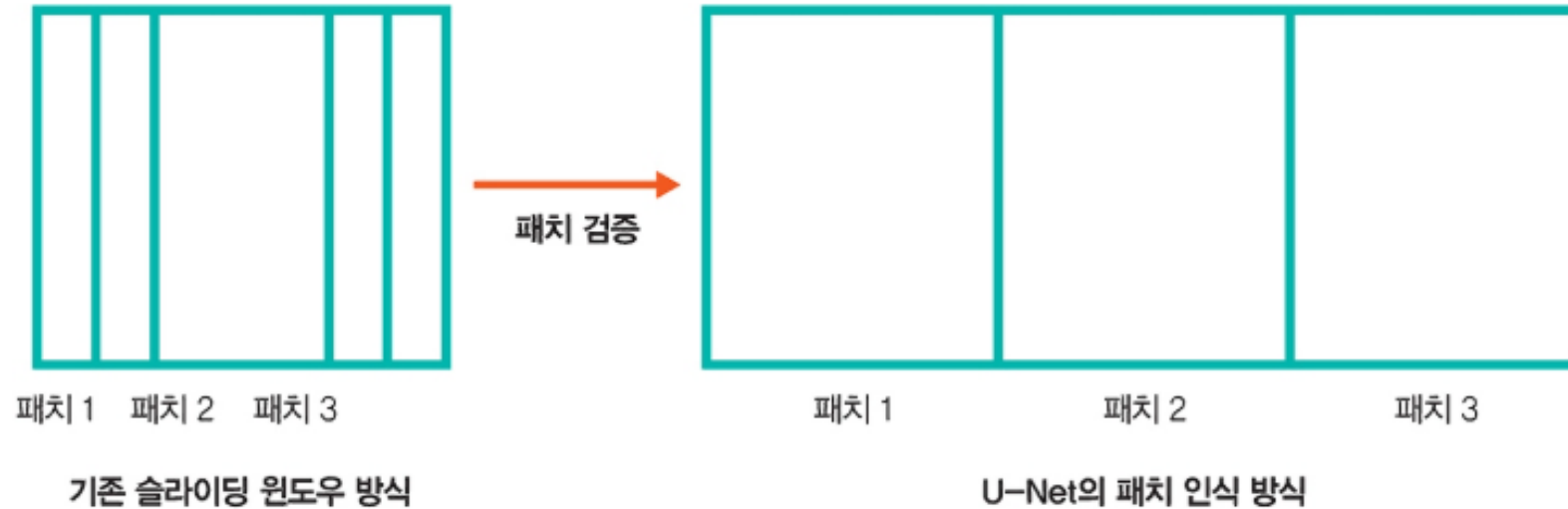


이때 합치는 과정에서 채널 방향으로 그대로 옆으로 갖다 붙이는 연산 (Concatenation)을 하기에 이로 인해 채널 수가 두 배로 증가

이후 3*3 합성곱 연산을 두 번씩 진행하여 채널의 크기를 4분의 1로 줄임
따라서 각 디코더 블록은 한 번의 Skip Connection 연산과
두 번의 합성곱 연산을 거치며 최종적으로 채널이 1/2배 크기로 줄어들게 됨

디코더가 아무리 특징 맵의 크기를 다시 키워내도, 이미 인코더에서 압축되며 잃어버린 정밀한 정보를 복원할 수는 없음.
따라서 각 인코더 블록마다 압축하기 전에 가지고 있던 특징 정보를 그대로 가져와
디코더 블록과 합쳐서 정보의 특징을 보존하는 기법

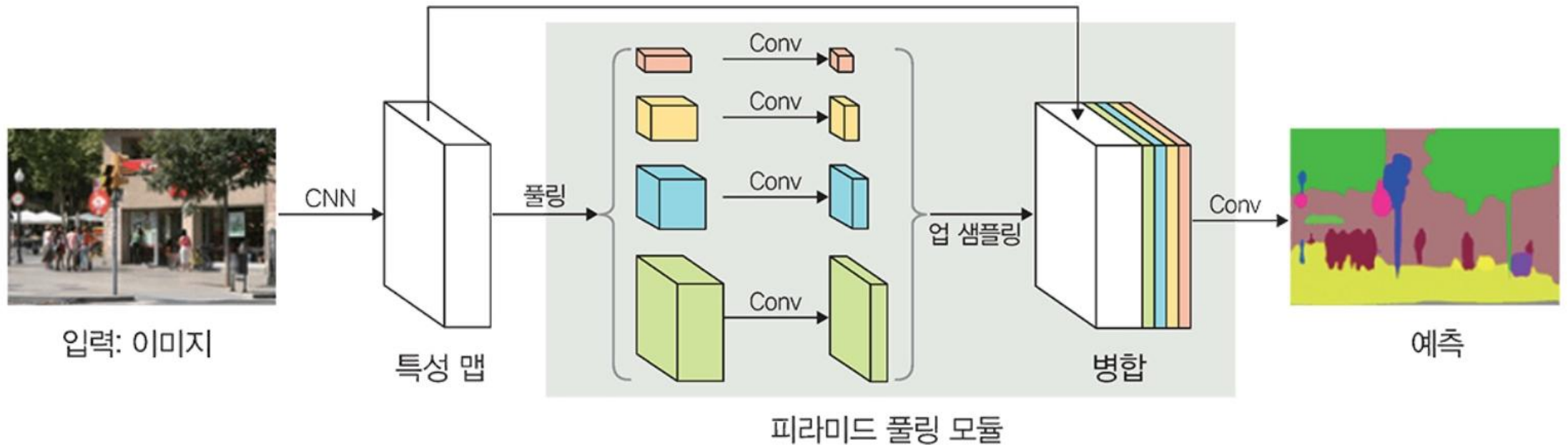
U-Net 슬라이딩 윈도우 방식



기존 슬라이딩 윈도우 방식은 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증하게 되어 속도가 느려짐

그러나 U-Net에서는 검증이 끝난 영역은 건너뛰는 패치 방식을 사용해 속도가 기존 연구들에 비해 속도가 빠름

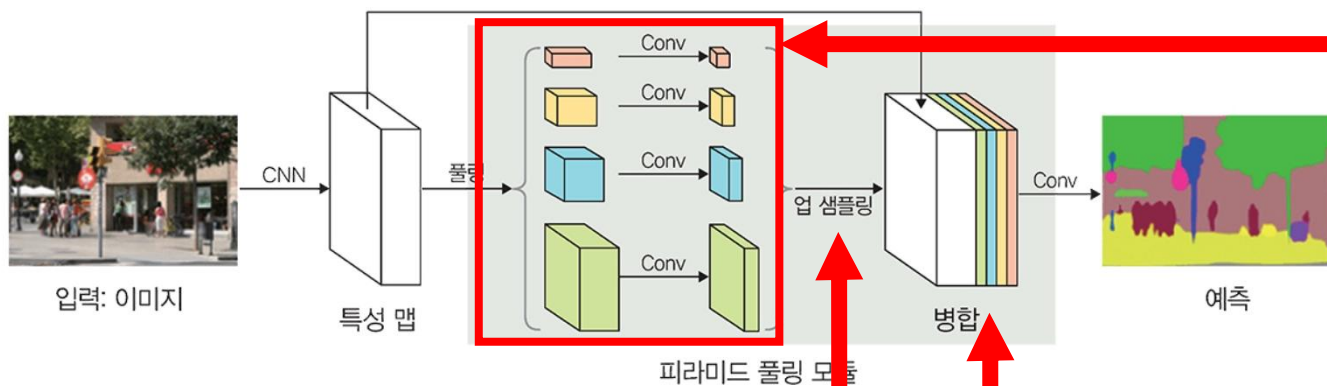
PSPNet 소개



PSPNet은 2017년에 발표된 시멘틱 분할(Semantic Segmentation, 의미론적 분할) 알고리즘

완전연결층의 한계를 극복하기 위해 **피라미드 풀링 모듈**을 추가한 구성을 가짐

PSPNet 구조



수행하는 풀링의 개수 등 설정은
구현에 따라 자유롭게 수정할 수 있음

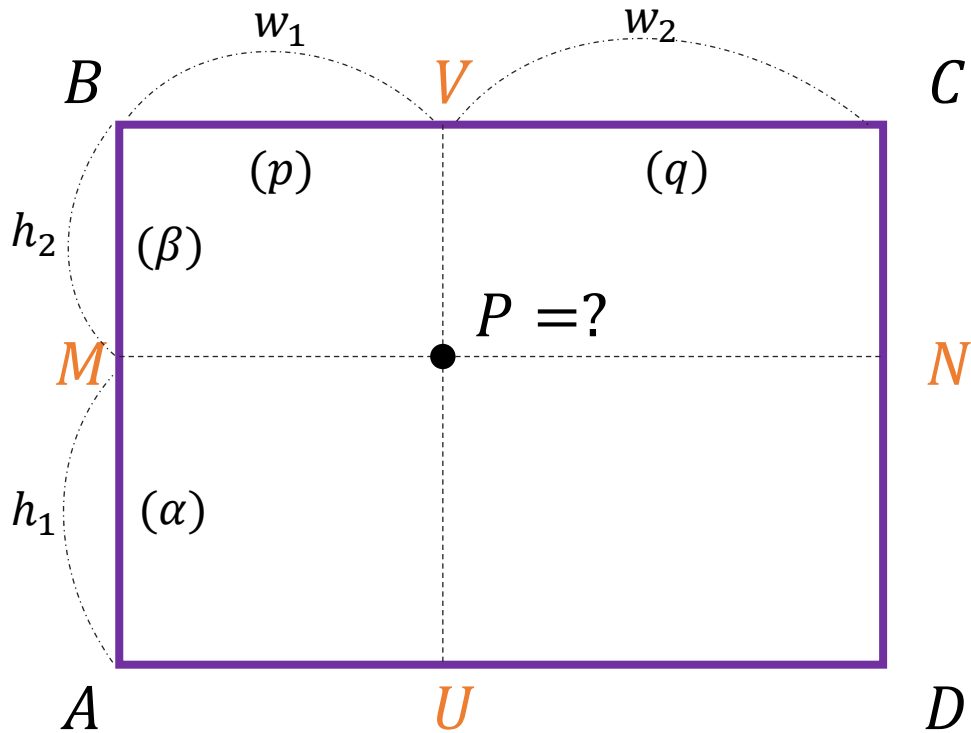
1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링 수행
2. 이때 1×1 , 2×2 , 3×3 , 6×6 크기로 풀링을 수행
3. 1×1 크기의 특성 맵이 가장 광범위하게 풀링하여 정보 추출
4. 나머지 크기의 특성 맵들은 서로 다른 영역의 정보를 추출

5. 이후 1×1 합성곱을 사용하여 채널 수를 조정
6. 풀링층의 개수가 N 일 때 출력 채널 수 = 입력 채널 수 / N
7. 이후 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링 진행
이때 **양선형 보간법**을 사용하여 진행

8. 원래의 특성 맵과 새롭게 생성한 각 특성 맵을 병합

PSPNet이 가지는 가장 큰 차이점은 기존 연구들과 같이
고정된 영역들 안에서 이미지를 분석하고 분할하는 것 뿐 아니라
 1×1 특성 맵이 가지는 전역적인 문맥(Context)과 다양한 크기의 특성 맵이 보이는 지역적 특징을 모두 고려하여
내부 개체를 더욱 정확히 예측하고 분할할 수 있게 한 점에 있음

양선형 보간법이란?



점 P에서 x축 방향으로 거리를 w_1 , w_2 라고 하며,
Y축 방향으로 거리를 h_1 , h_2 라고 하며,
알고 있는 주변 네 점의 데이터 값을 A, B, C, D 라 하면

점 P의 값을 양선형 보간법으로 계산 시

$$P = q(\beta A + \alpha B) + p(\beta D + \alpha C)$$

즉, 각 점 A, B, C, D에 대해
가로와 세로에 대한 가중치를 곱한 뒤 합산하여 점 P를 획득

보간법이란 빈 화소에 값을 할당하여 이전보다 높은 품질로 영상을 향상하는 방법을 의미
선형 보간법은 원본 영상의 화소 값 두개를 사용하여 원하는 좌표에서 새로운 화소 값을 계산

양선형 보간법은 화소당 선형 보간을 세번 수행하며,
새롭게 생성된 화소는 가장 가까운 화소 4개에 가중치를 곱한 값을 합산해 얻음

DeepLabv3 / DeepLabv3+ 소개

인코더

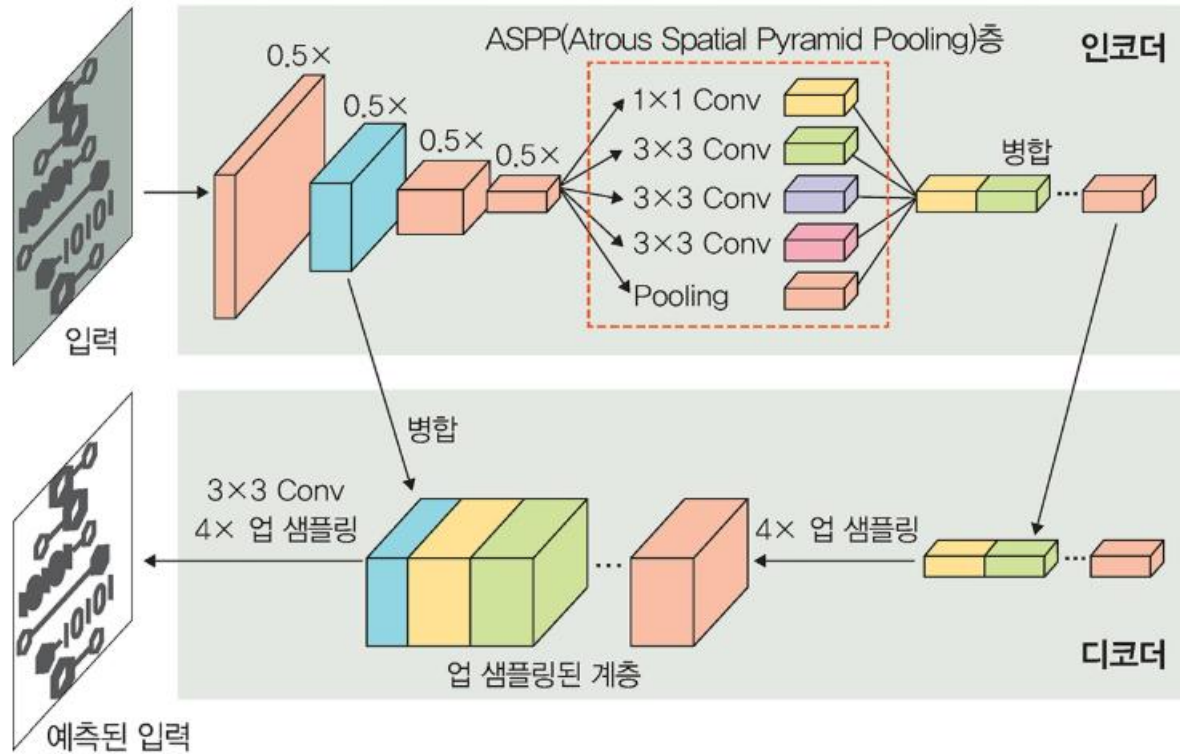
PSPNet의 피라미드 풀링 모듈과 유사한 구조로
ASPP(Atrous Spatial Pyramid Pooling) 층을 두어

여러 크기의 Conv를 통과 시킨 뒤
이를 기반으로 다양한 특징 맵을 추출 후 병합

디코더

인코더의 특징 맵을 업샘플링하며 입력 이미지와 유사한
해상도로 변환

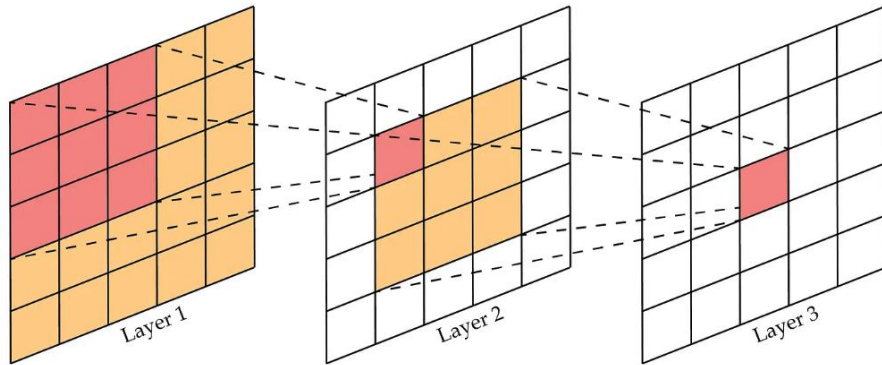
이때 이전 인코더의 입력 이미지로부터 얻은 특징 데이터를
한번 더 출력 전에 병합하여 높은 품질의 특징 맵을 가지게 함



DeepLabv3 / DeepLabv3+ 역시 완전연결층의 단점을 보완하기 Atrous 합성곱을 사용하는 구조
인코더-디코더 구조를 사용하지만, 인코더-디코더 구조에서는 조절하기 어렵던 특성 맵 해상도를
인코더에 Atrous 합성곱을 도입하여 제어하는 데 성공한 네트워크

수용 영역이란?

Receptive Field in Convolutional Networks



수용 영역이 크면 전역적(Global)인 맥락을 분석하는데 유용

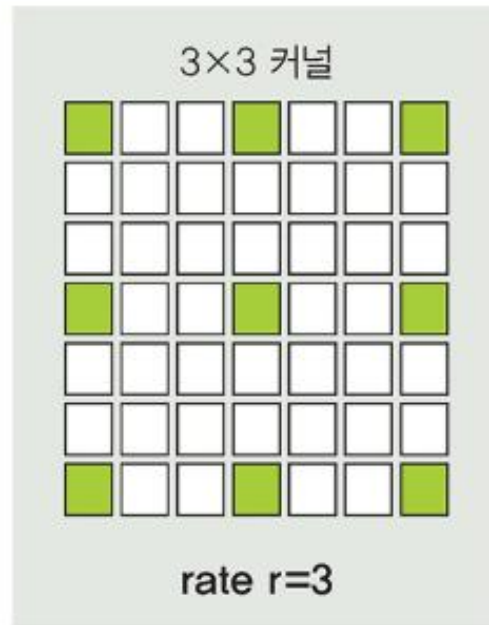
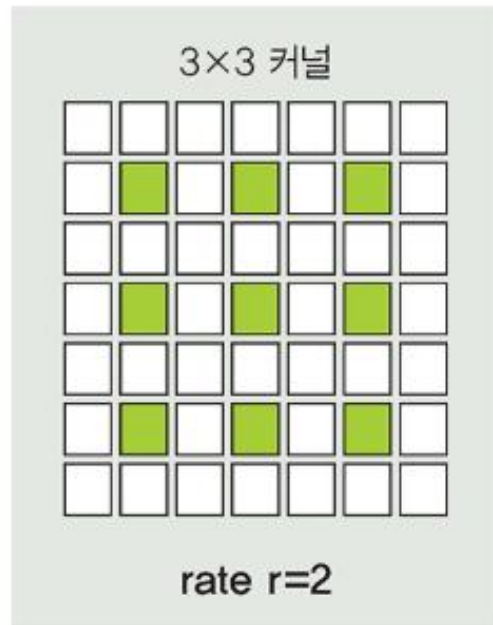
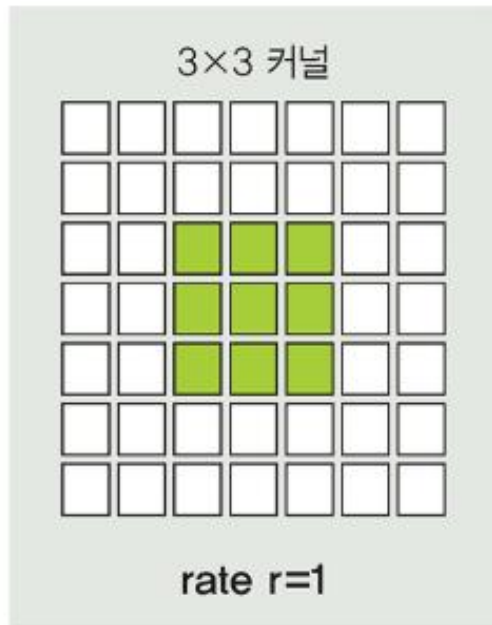
수용 영역이 작으면 국소적(Local)인 특징을 분석하는데 유용



수용 영역이란 외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미
즉, 영상에서 특정 위치에 있는 픽셀들은 그 주변에 있는 일부 픽셀과 가까울 수록 연관성이 높고 영향이 큼

영상 전체 영역이 아닌 특정 범위를 한정해서 처리하는 방식이 **수용 영역**

Atrous 합성곱이란?



Atrous 합성곱
필터 내부에 빈 공간을 둔 채로 작동하는
합성곱 방식을 의미

Rate 값을 조절하여 얼마나 많은
빈 공간을 가질 지 제어할 수 있음

Rate가 1이면 일반 합성곱과
동일한 연산을 수행하며,
rate가 커질 수록 빈 공간은 더 커짐

이미지 분할에서 높은 성능을 내려면 수용 영역(Receptive Field)의 크기가 중요
Atrous 합성곱 사용시 기존 합성곱에 비해 파라미터 수를 늘리지 않으면서도
수용영역을 크게 키울 수 있어 이미지 분할 분야에서 많이 사용

U-Net 실습

▶ # 필요한 패키지 설치
!pip install -q segmentation-models-pytorch albumentations

```
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
from torchvision.datasets import OxfordIIITPet

import segmentation_models_pytorch as smp

# 재현성을 위한 시드 고정
torch.manual_seed(42)
np.random.seed(42)

# GPU 사용 가능 여부 확인
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'사용 장치: {device}')
if torch.cuda.is_available():
    print(f'GPU 모델: {torch.cuda.get_device_name(0)}')
```

필요 라이브러리 설치

랜덤으로 결과가 바뀌는 것을 막기 위해 시드 고정

GPU 사용 가능 여부 및 사용 GPU 확인

하이퍼 파라미터 설정

```
# 하이퍼 파라미터 설정
IMAGE_SIZE      = 256      # 입력 이미지 크기 (256x256)
BATCH_SIZE      = 8        # 배치 크기 (GPU 메모리 부족 시 4로 줄이세요)
NUM_EPOCHS      = 1        # 학습 에폭 수 (더 좋은 결과는 20~30)
LEARNING_RATE    = 1e-4    # 학습률
NUM_CLASSES     = 3        # 클래스 수: 전경 / 배경 / 경계
DATA_DIR        = './data'
```

U-Net 실습

```
class PetSegmentationDataset(Dataset):
    """
    Oxford-IIIT Pet Dataset 세그멘테이션용 커스텀 데이터셋.
    torchvision의 OxfordIIITPet을 내부적으로 사용합니다.
    """
    def __init__(self, root, split='trainval', image_size=256, download=True):
        self.dataset = OxfordIIITPet(
            root=root,
            split=split,
            target_types='segmentation', # 픽셀 단위 마스크 사용
            download=download
        )
        self.image_size = image_size

        # 이미지 전처리: 리사이즈 -> 텐서 변환 -> ImageNet 정규화
        self.img_transform = T.Compose([
            T.Resize((image_size, image_size)),
            T.ToTensor(),
            T.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
        ])

        # 최근접 이웃 보간
        self.mask_transform = T.Resize(
            (image_size, image_size),
            interpolation=T.InterpolationMode.NEAREST
        )
```

반려동물 데이터가 들어 있는
Oxford-IIIT Pet Dataset을 사용

Oxford-IIIT Pet Dataset의 세그멘테이션 마스크는 3가지
값을 가짐:

- 1: 전경 (반려동물 영역)
- 2: 배경
- 3: 경계 (윤곽선 부근)

학습 편의상 0-indexed로 변환합니다:
0(전경), 1(배경), 2(경계)

이미지는 이전의 하이퍼 파라미터에서 정한
256*256으로 데이터셋 이미지를 리사이즈

U-Net 실습

```
def __len__(self):  
    return len(self.dataset)  
  
def __getitem__(self, idx):  
    image, mask = self.dataset[idx]  
    image = self.img_transform(image)  
    mask = self.mask_transform(mask)  
    mask = torch.tensor(np.array(mask), dtype=torch.long)  
    mask = mask - 1  
    return image, mask
```

```
print('데이터셋 다운로드 중... (첫 실행 시 약 800MB, 수 분 소요)')  
train_dataset = PetSegmentationDataset(DATA_DIR, split='trainval', image_size=IMAGE_SIZE)  
test_dataset = PetSegmentationDataset(DATA_DIR, split='test', image_size=IMAGE_SIZE)  
  
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)  
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)  
  
print(f'학습 샘플 수 : {len(train_dataset)}')  
print(f'테스트 샘플 수 : {len(test_dataset)}')
```

데이터 로더 정의

최초 실행 시 데이터셋을 다운로드

이후 학습 샘플수와 테스트 샘플 수를 표시

데이터셋 다운로드 중... (첫 실행 시 약 800MB, 수 분 소요)

학습 샘플 수 : 3680

테스트 샘플 수 : 3669

U-Net 실습

```
# 시각화 헬퍼 함수
def denormalize(tensor):
    """ImageNet 정규화 역변환 (시각화용)"""
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)
    return (tensor * std + mean).clamp(0, 1)

# 전경(주황), 배경(파랑), 경계(노랑)
COLORS = np.array([[255, 100, 80], [80, 160, 255], [255, 230, 50]], dtype=np.uint8)
CLASS_NAMES = ['전경 (동물)', '배경', '경계']

def mask_to_rgb(mask_tensor):
    """정수 마스크(H,W) -> RGB 이미지(H,W,3)"""
    mask_np = np.clip(mask_tensor.numpy(), 0, NUM_CLASSES - 1)
    return COLORS[mask_np]

# 데이터 샘플 시각화
images, masks = next(iter(train_loader))
fig, axes = plt.subplots(3, 4, figsize=(14, 10))
fig.suptitle('Oxford-IIIT Pet Dataset 샘플 확인', fontsize=14, fontweight='bold')

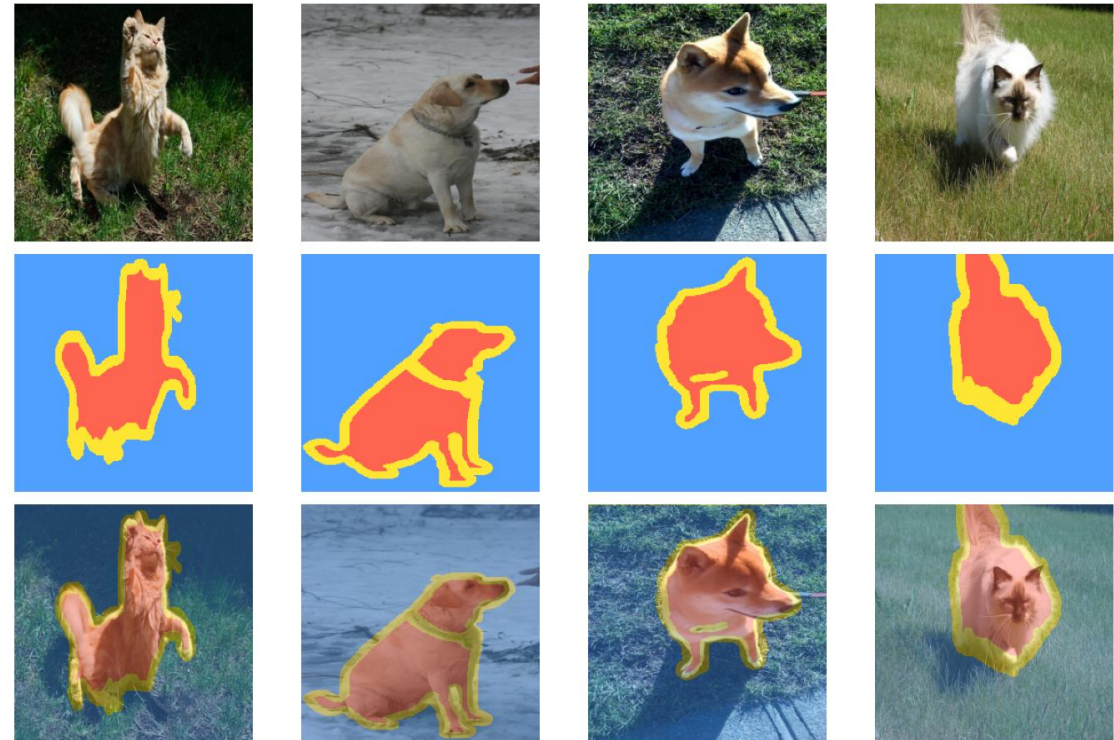
row_labels = ['원본 이미지', '정답 마스크', '오버레이']
for row, label in enumerate(row_labels):
    axes[row, 0].set_ylabel(label, fontsize=11)

for i in range(4):
    img_np = denormalize(images[i]).permute(1, 2, 0).numpy()
    mask_rgb = mask_to_rgb(masks[i])
    overlay = img_np * 0.6 + mask_rgb / 255.0 * 0.4
    axes[0, i].imshow(img_np); axes[0, i].axis('off')
    axes[1, i].imshow(mask_rgb); axes[1, i].axis('off')
    axes[2, i].imshow(overlay); axes[2, i].axis('off')

patches = [mpatches.Patch(color=COLORS[c]/255, label=CLASS_NAMES[c]) for c in range(NUM_CLASSES)]
fig.legend(handles=patches, loc='lower center', ncol=3, fontsize=11, bbox_to_anchor=(0.5, 0.01))
plt.tight_layout(rect=[0, 0.05, 1, 1])
plt.show()
```

시각화 함수 정의 및

데이터의 샘플을 시각화하여 우선 확인



U-Net 실습

```
# U-Net 모델 생성
# encoder_name      : 인코더 백본 (VGG16, ImageNet 사전학습)
# encoder_weights   : 사전학습 가중치 ('imagenet')
# in_channels       : 입력 채널 (RGB = 3)
# classes           : 출력 클래스 수 (전경/배경/경계 = 3)
model = smp.Unet(
    encoder_name      = 'vgg16',
    encoder_weights   = 'imagenet',
    in_channels       = 3,
    classes           = NUM_CLASSES,
)
model = model.to(device)

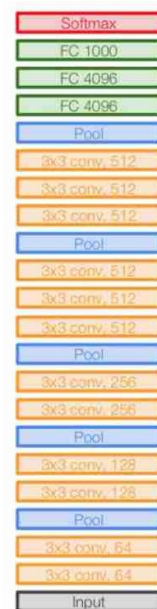
total_params        = sum(p.numel() for p in model.parameters())
trainable_params    = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'전체 파라미터 수 : {total_params:,}')
print(f'학습 가능 파라미터: {trainable_params:,}')
```

전체 파라미터 수 : 23,748,531
학습 가능 파라미터: 23,748,531

U-Net 자체 인코더를 사용하면 시간이 많이 걸려
이번 실습에서만 인코더 부분을 VGG16을 사용

해당 VGG16은 ImageNet에서 사전학습한
사전학습 가중치를 사용

이후 전체 파라미터와 학습 가능 파라미터 확인



VGG16

U-Net 실습

```
# CrossEntropyLoss: 다중 클래스 픽셀 분류에 가장 일반적
criterion = nn.CrossEntropyLoss(ignore_index=255)
```

```
# Adam: 세그멘테이션 태스크에서 널리 사용
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

```
# ReduceLROnPlateau: 검증 손실 정체시 학습률 절반으로 감소
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=2
)
```

```
def compute_iou(preds, labels, num_classes=NUM_CLASSES, ignore_index=255):
    """
    mean IoU (Intersection over Union) 계산
    세그멘테이션 품질의 표준 평가 지표
    
$$IoU = TP / (TP + FP + FN)$$

    """
    iou_list = []
    preds = preds.view(-1)
    labels = labels.view(-1)
    valid = labels != ignore_index
    preds, labels = preds[valid], labels[valid]
    for cls in range(num_classes):
        pred_c = preds == cls
        label_c = labels == cls
        inter = (pred_c & label_c).sum().float()
        union = (pred_c | label_c).sum().float()
        if union == 0:
            continue
        iou_list.append((inter / union).item())
    return np.mean(iou_list) if iou_list else 0.0
```

다중 클래스를 분류하기 위해 Cross Entropy Loss 사용

Adam 옵티마이저 사용

표준적인 평가 지표인 평균 IoU를 계산하는 함수 정의

U-Net 실습

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    """한 에폭 학습용 코드, 이걸 반복해서 학습하는 것"""
    model.train()
    total_loss = 0.0
    total_iou = 0.0
    for batch_idx, (images, masks) in enumerate(loader):
        images = images.to(device)      # (B, 3, H, W)
        masks = masks.to(device)        # (B, H, W)
        outputs = model(images)          # (B, num_classes, H, W)
        loss = criterion(outputs, masks)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        preds = outputs.argmax(dim=1)
        batch_iou = compute_iou(preds.cpu(), masks.cpu())
        total_loss += loss.item()
        total_iou += batch_iou
        if (batch_idx + 1) % 10 == 0:
            print(f'  배치 [{batch_idx+1}/{len(loader)}]   '
                  f'손실: {loss.item():.4f}   IoU: {batch_iou:.4f}')
    return total_loss / len(loader), total_iou / len(loader)
```

하나의 학습 사이클 단위인 에포크(Epoch)의 연산 과정을 정의

입력 이미지의 형태는 (B, 3, H, W)이고, 정답 마스크는 (B, H, W) 형태임, B: Batch, 3: 채널임

모델은 입력 이미지를 받아 (B, num_classes, H, W) 형태의 출력을 생성하며, 이는 각 픽셀에 대해 전경, 배경, 경계 등 각 클래스에 속할 점수를 출력함

예측 결과와 정답 마스크를 비교하여 손실값을 계산

즉, U-Net이 각 픽셀을 전경, 배경, 경계 중 무엇으로 예측했는지 확인하고, 그 예측이 정답과 얼마나 다른지를 숫자로 계산하는 과정을 나타내며, 이 손실값을 줄이도록 모델이 학습함

argmax를 이용해 각 픽셀에서 가장 높은 점수를 가진 클래스를 최종 예측값으로 선택하고, 이를 정답 마스크와 비교하여 IoU를 계산함

U-Net 실습

```
@torch.no_grad()
def evaluate(model, loader, criterion, device):
    """검증/테스트 평가"""
    model.eval()
    total_loss = 0.0
    total_iou = 0.0
    for images, masks in loader:
        images = images.to(device)
        masks = masks.to(device)
        outputs = model(images)
        loss = criterion(outputs, masks)
        preds = outputs.argmax(dim=1)
        total_loss += loss.item()
        total_iou += compute_iou(preds.cpu(), masks.cpu())
    return total_loss / len(loader), total_iou / len(loader)
```

학습 성능을 검증하고 평가하는 Evaluate 함수를 정의

이미지와 정답 마스크를 배치 단위로 불러오고, 모델 출력값과 정답 마스크를 비교하여 손실값을 계산함

`argmax(dim=1)`으로 각 픽셀의 최종 예측 클래스를 선택하고, 정답 마스크와 비교하여 평균 손실과 평균 IoU를 반환함

U-Net 실습

```
history = {'train_loss': [], 'test_loss': [], 'train_iou': [], 'test_iou': []}
best_iou = 0.0

print('=' * 60)
print(f' U-Net 학습 시작 | 총 {NUM_EPOCHS} 에폭 | 장치: {device}')
print('=' * 60)

for epoch in range(1, NUM_EPOCHS + 1):
    print(f' \n[에폭 {epoch}/{NUM_EPOCHS}]')
    train_loss, train_iou = train_one_epoch(model, train_loader, optimizer, criterion, device)
    test_loss, test_iou = evaluate(model, test_loader, criterion, device)
    scheduler.step(test_loss)
    history['train_loss'].append(train_loss)
    history['test_loss'].append(test_loss)
    history['train_iou'].append(train_iou)
    history['test_iou'].append(test_iou)
    if test_iou > best_iou:
        best_iou = test_iou
        torch.save(model.state_dict(), 'best_unet_pet.pth')
        print(f' 성능 최고 모델을 저장 (test IoU: {best_iou:.4f})')
    print(f' 학습 -- 손실: {train_loss:.4f} | mIoU: {train_iou:.4f}')
    print(f' 테스트 -- 손실: {test_loss:.4f} | mIoU: {test_iou:.4f}')

print('\n학습 완료!')
print(f'최고 테스트 mIoU: {best_iou:.4f}')
```

```
배치 [370/460] 손실: 0.1011 IoU: 0.8529
배치 [380/460] 손실: 0.1102 IoU: 0.8673
배치 [390/460] 손실: 0.1242 IoU: 0.8385
배치 [400/460] 손실: 0.1395 IoU: 0.7925
배치 [410/460] 손실: 0.2242 IoU: 0.7506
배치 [420/460] 손실: 0.1432 IoU: 0.8110
배치 [430/460] 손실: 0.2292 IoU: 0.7477
배치 [440/460] 손실: 0.1506 IoU: 0.7920
배치 [450/460] 손실: 0.1929 IoU: 0.7836
배치 [460/460] 손실: 0.2376 IoU: 0.7637
학습 -- 손실: 0.1237 | mIoU: 0.8400
테스트 -- 손실: 0.2249 | mIoU: 0.7752
```

학습 완료!
최고 테스트 mIoU: 0.8100

학습 및 테스트 진행

진행 중 배치와 손실, IoU 변화를 로그로 출력

U-Net 실습

```
epochs = range(1, NUM_EPOCHS + 1)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle('U-Net 학습 곡선', fontsize=13, fontweight='bold')

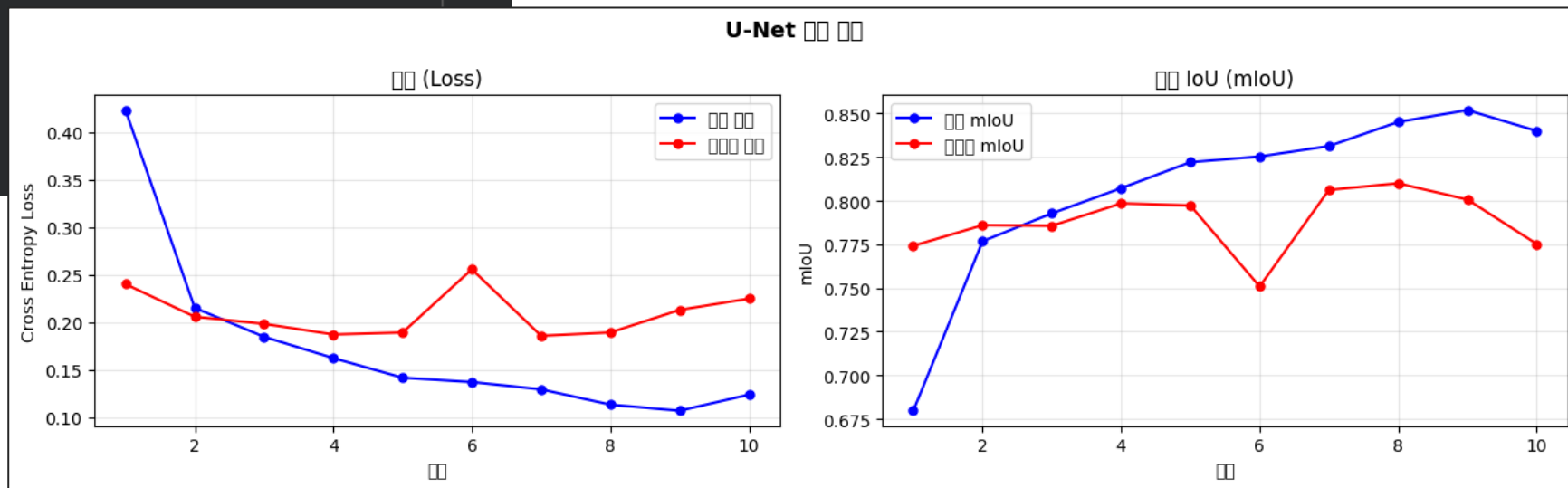
ax1.plot(epochs, history['train_loss'], 'b-o', label='학습 손실', markersize=5)
ax1.plot(epochs, history['test_loss'], 'r-o', label='테스트 손실', markersize=5)
ax1.set_title('손실 (Loss)')
ax1.set_xlabel('에폭')
ax1.set_ylabel('Cross Entropy Loss')
ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(epochs, history['train_iou'], 'b-o', label='학습 mIoU', markersize=5)
ax2.plot(epochs, history['test_iou'], 'r-o', label='테스트 mIoU', markersize=5)
ax2.set_title('평균 IoU (mIoU)')
ax2.set_xlabel('에폭')
ax2.set_ylabel('mIoU')
ax2.legend(); ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Epoch 마다 Loss와 mIoU의 변화를 그래프를 출력해 확인

Loss의 경우 지속 하락하다가 상승하거나 튀는 경우 발생
mIoU의 경우 상승하다가 상한 수렴하거나 줄어드는 경우 발생



U-Net 실습

학습 종료 후 임의의 데이터셋 내 샘플로 성능을 검증

개체의 영역과 경계가 분할된 결과

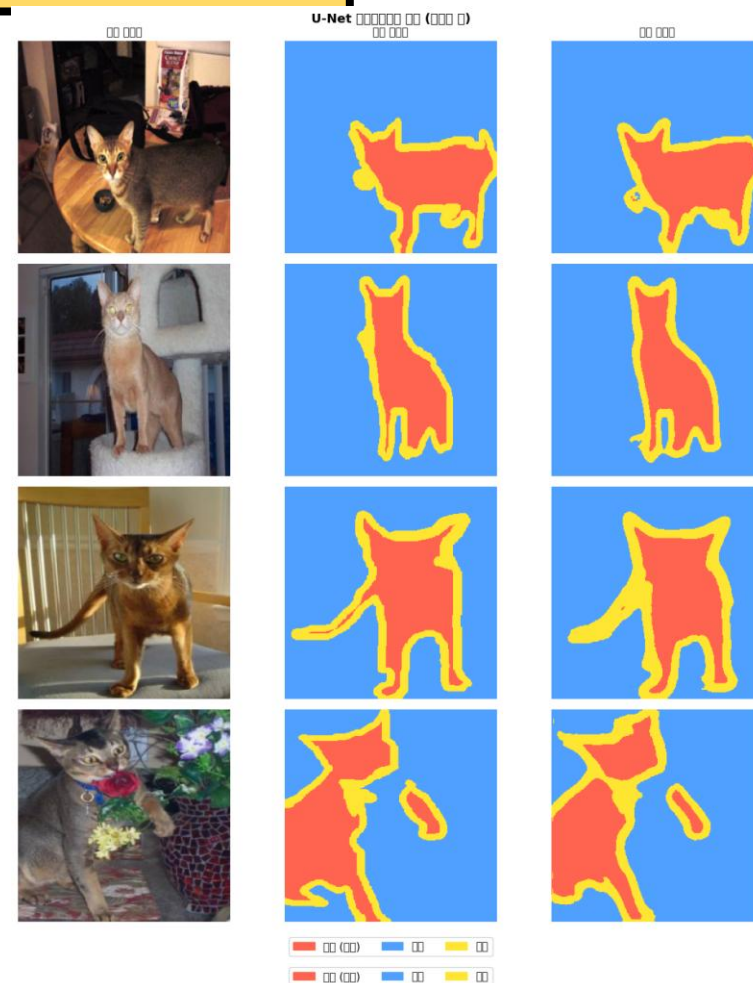
```
# 최고 성능 모델 로드
model.load_state_dict(torch.load('best_unet_pet.pth', map_location=device))
model.eval()
print('최고 성능 모델 로드 완료')

images, masks = next(iter(test_loader))
with torch.no_grad():
    outputs = model(images.to(device))
    preds = outputs.argmax(dim=1).cpu()

n_samples = min(4, BATCH_SIZE)
fig, axes = plt.subplots(n_samples, 3, figsize=(12, n_samples * 3.5))
fig.suptitle('U-Net 세그멘테이션 결과 (테스트 셋)', fontsize=13, fontweight='bold')
for ax, title in zip(axes[0], ['원본 이미지', '정답 마스크', '예측 마스크']):
    ax.set_title(title, fontsize=11, fontweight='bold')

for i in range(n_samples):
    img_show = denormalize(images[i]).permute(1, 2, 0).numpy()
    gt_show = mask_to_rgb(masks[i])
    pred_show = mask_to_rgb(preds[i])
    iou_val = compute_iou(preds[i].unsqueeze(0), masks[i].unsqueeze(0))
    axes[i, 0].imshow(img_show); axes[i, 0].axis('off')
    axes[i, 1].imshow(gt_show); axes[i, 1].axis('off')
    axes[i, 2].imshow(pred_show); axes[i, 2].axis('off')
    axes[i, 2].set_xlabel(f'IoU: {iou_val:.3f}', fontsize=10)

patches = [mpatches.Patch(color=COLORS[c]/255, label=CLASS_NAMES[c]) for c in range(NUM_CLASSES)]
fig.legend(handles=patches, loc='lower center', ncol=3, fontsize=11, bbox_to_anchor=(0.5, 0.01))
plt.tight_layout(rect=[0, 0.04, 1, 1])
plt.show()
```



왼쪽 열은 테스트 데이터의 원본 이미지, 가운데 열은 데이터셋에 포함된 정답 마스크, 오른쪽 열은 학습된 U-Net 모델이 예측한 마스크

U-Net 실습

```
final_loss, final_iou = evaluate(model, test_loader, criterion, device)
print("=" * 40)
print("    최종 테스트 성능")
print("=" * 40)
print(f" Cross Entropy Loss : {final_loss:.4f}")
print(f" mean IoU (mIoU)   : {final_iou:.4f} ({final_iou*100:.1f}%)")
print("=" * 40)

# 클래스별 IoU
intersection = torch.zeros(NUM_CLASSES)
union        = torch.zeros(NUM_CLASSES)

model.eval()
with torch.no_grad():
    for images, masks in test_loader:
        outputs = model(images.to(device))
        preds   = outputs.argmax(dim=1).cpu().view(-1)
        labels  = masks.view(-1)
        valid   = labels != 255
        preds, labels = preds[valid], labels[valid]
        for cls in range(NUM_CLASSES):
            pred_c = preds == cls
            label_c = labels == cls
            intersection[cls] += (pred_c & label_c).sum().float()
            union[cls]       += (pred_c | label_c).sum().float()

print("클래스별 IoU:")
for cls in range(NUM_CLASSES):
    iou = (intersection[cls] / union[cls]).item() if union[cls] > 0 else 0.0
    print(f" {CLASS_NAMES[cls]:<15}: {iou:.4f} ({iou*100:.1f}%)")
```

최종 테스트 성능 및 전경, 배경, 경계의 IoU 출력
경계의 IoU가 다소 낮은 걸 확인할 수 있음

```
=====
              최종 테스트 성능
=====
Cross Entropy Loss : 0.1892
mean IoU (mIoU)   : 0.8100 (81.0%)
=====

클래스별 IoU:
전경 (동물)       : 0.8827 (88.3%)
배경               : 0.9392 (93.9%)
경계               : 0.6104 (61.0%)
```

U-Net 실습

```
from google.colab import files

def predict_single_image(image_path, model, device, image_size=IMAGE_SIZE):
    transform = T.Compose([
        T.Resize((image_size, image_size)),
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
    img = Image.open(image_path).convert('RGB')
    inp = transform(img).unsqueeze(0).to(device)
    model.eval()
    with torch.no_grad():
        pred = model(inp).argmax(dim=1)[0].cpu()
    img_show = img.resize((image_size, image_size))
    pred_rgb = mask_to_rgb(pred)
    overlay = np.array(img_show) / 255.0 * 0.6 + pred_rgb / 255.0 * 0.4
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    fig.suptitle('단일 이미지 세그멘테이션 결과', fontsize=13, fontweight='bold')
    axes[0].imshow(img_show); axes[0].set_title('원본'); axes[0].axis('off')
    axes[1].imshow(pred_rgb); axes[1].set_title('예측 마스크'); axes[1].axis('off')
    axes[2].imshow(overlay); axes[2].set_title('오버레이'); axes[2].axis('off')
    patches = [mpatches.Patch(color=COLORS[c]/255, label=CLASS_NAMES[c]) for c in range(NUM_CLASSES)]
    fig.legend(handles=patches, loc='lower center', ncol=3, fontsize=11)
    plt.tight_layout(rect=[0, 0.08, 1, 1])
    plt.show()

print('이미지 파일을 업로드하세요 (jpg, png 등)')
uploaded = files.upload()
for filename in uploaded.keys():
    print(f'{filename} 추론 중...')
    predict_single_image(filename, model, device)
```

이미지 파일을 업로드하세요 (jpg, png 등)

파일 선택

선택된 파일 없음

Cancel upload

자신이 원하는 임의의 이미지를 집어넣고
학습을 마친 모델에 바로 적용하여
결과를 확인할 수 있음

